DOCUMENTACION TECNICA COMPLETA

Business Management CRUD

Sistema de Gestion Empresarial Multi-Compania

Python FastAPI Rich Typer JSON Storage uv

Campo	Valor
Lenguaje	Python 3.12+
API Backend	FastAPI
Interfaz de Usuario	Typer (CLI) + Rich (tablas y paneles en terminal)
Almacenamiento	JSON (built-in json module)
Patron	API REST + CLI cliente (100 % Python puro)
Autenticacion	hashlib SHA-256 + JWT (PyJWT)
Exportacion	CSV (built-in csv module)
Gestor de paquetes	uv
Version Doc.	2.1.0

Toda la interfaz de usuario se implementa en Python puro con Typer y Rich. No existe ningun archivo HTML, CSS ni JavaScript en el proyecto.

1. Vision General del Sistema

Business Management CRUD es un sistema compuesto por dos capas 100 % Python: un backend REST con FastAPI que expone la logica de negocio y los datos, y una interfaz de linea de comandos (CLI) construida con Typer y Rich que consume dicha API. Todo el proyecto se gestiona y ejecuta con uv.

1.1 Objetivo Principal

Proveer a pequenas y medianas empresas una herramienta local (localhost), sin dependencias de nube, sin navegador web y sin ningun codigo frontend, para administrar empleados, inventario, clientes, ventas y reportes. La interfaz completa se ejecuta en la terminal.

1.2 Principios de Diseno

- 100 % Python: toda la UI es codigo Python puro usando Typer y Rich.
- Modularidad: cada funcionalidad es un router FastAPI + comando Typer independiente.
- Portabilidad: corre en Windows, macOS y Linux con uv sin conflictos de entornos.
- Persistencia local: todos los datos en archivos JSON del directorio del proyecto.
- Seguridad basica: contrasenas hasheadas con SHA-256; sesiones con JWT (PyJWT).
- Escalabilidad: agregar modulos nuevos solo requiere un nuevo router + comando Typer.

1.3 Arquitectura de Dos Capas

El sistema se divide en dos procesos Python que se comunican por HTTP:

Capa	Proceso	Tecnologia	Descripcion
Backend	uv run serve	FastAPI + uvicorn	API REST: datos, logica de negocio y autenticacion JWT.
Frontend	uv run bm	Typer + Rich	CLI: llama a la API con httpx y renderiza la respuesta en terminal.

El usuario solo interactua con la CLI. El servidor FastAPI corre en segundo plano en localhost:8000.

2. Arquitectura del Sistema

2.1 Estructura de Carpetas

```
business_manager/
|-- main.py                   # Arranca FastAPI con uvicorn
|-- cli.py                    # Punto de entrada de la CLI (Typer app raiz)
|-- config.py                 # Constantes globales (rutas, JWT, URL de la API)
|-- pyproject.toml            # Dependencias y scripts gestionados por uv
|-- uv.lock                   # Lock file de uv
|-- data/                     # Datos persistentes
|   |-- companies.json
|   `-- {company_id}/
|       |-- company.json
|       |-- employees.json
|       |-- products.json
|       |-- clients.json
|       `-- orders.json
|-- core/
|   |-- auth.py               # Logica de login, registro, JWT
|   |-- data_manager.py       # CRUD generico sobre archivos JSON
|   `-- session.py            # Depends FastAPI: get_current_company
|-- models/                   # Clases Pydantic
|   |-- company.py
|   |-- employee.py
|   |-- product.py
|   |-- client.py
|   `-- order.py
|-- routers/                  # Endpoints FastAPI
|   |-- auth_router.py
|   |-- employee_router.py
|   |-- product_router.py
|   |-- client_router.py
|   |-- order_router.py
|   `-- report_router.py
|-- services/                 # Logica de negocio
|   |-- auth_service.py
|   |-- employee_service.py
|   |-- product_service.py
|   |-- client_service.py
|   `-- order_service.py
|-- commands/                 # Comandos Typer (UI en terminal con Rich)
|   |-- auth_cmd.py
|   |-- employee_cmd.py
|   |-- product_cmd.py
|   |-- client_cmd.py
|   |-- order_cmd.py
|   `-- report_cmd.py
`-- utils/
    |-- validators.py
    |-- formatters.py
    |-- id_generator.py
```

```
    |-- token.py              # Lee/escribe ~/.bm_token (JWT local)
    `-- exporter.py
```

2.2 Tabla de Capas

Capa	Carpeta	Tecnologia	Responsabilidad
Model	models/	Pydantic	Define y valida la estructura de datos.
View	commands/	Typer + Rich	Interaccion con el usuario: tablas, paneles, prompts en terminal.
Router	routers/	FastAPI	Endpoints REST; recibe requests HTTP, devuelve JSON.
Service	services/	Python puro	Logica de negocio, validaciones, orquestacion.
Data	core/	json built-in	CRUD generico sobre archivos JSON con threading.Lock.
Utils	utils/	Python puro	Validadores, formateadores, generador de IDs, exportador CSV.

3. Estructura de Datos (JSON)

Toda la persistencia se realiza con archivos JSON usando el modulo built-in de Python. Los modelos Pydantic validan los datos al entrar y salir de los endpoints FastAPI.

3.1 companies.json

```
[{
  "id":         "comp_001",
  "name":       "Tech Solutions S.A.",
  "email":      "admin@techsolutions.com",
  "password":   "<sha256_hash>",
  "created_at": "2024-01-15T10:30:00",
  "active":     true
}]
```

3.2 Otros archivos JSON (company, employees, products, clients, orders)

Los esquemas de company.json, employees.json, products.json, clients.json y orders.json son identicos a la version original (ver seccion 3 de la documentacion v1.0). Los campos no cambian; la diferencia es que ahora son validados automaticamente por modelos Pydantic al pasar por los endpoints FastAPI.

4. Backend FastAPI

4.1 main.py — Punto de Entrada del Servidor

```
from fastapi import FastAPI
from routers import (auth_router, employee_router, product_router,
                     client_router, order_router, report_router)

app = FastAPI(title='Business Manager', version='2.1.0')

app.include_router(auth_router.router,     prefix='/auth')
app.include_router(employee_router.router, prefix='/employees')
app.include_router(product_router.router,  prefix='/products')
app.include_router(client_router.router,   prefix='/clients')
app.include_router(order_router.router,    prefix='/orders')
app.include_router(report_router.router,   prefix='/reports')

def start():
    import uvicorn
    uvicorn.run('main:app', host='127.0.0.1', port=8000, reload=True)
```

4.2 Autenticacion JWT (core/auth.py + routers/auth_router.py)

Metodo	Ruta	Descripcion
POST	/auth/register	Registra empresa nueva; crea directorios y archivos JSON vacios.
POST	/auth/login	Valida credenciales SHA-256; retorna JWT con company_id y expiracion.
POST	/auth/logout	Registra el logout (el cliente descarta el token localmente).
GET	/auth/companies	Retorna lista de nombres de empresas registradas (para el prompt de login).

La dependencia get_current_company en core/session.py extrae y valida el JWT del header Authorization: Bearer <token>. Todos los routers protegidos la reciben via Depends().

4.3 Endpoints Estandar por Modulo

Metodo	Ruta	Descripcion
GET	/{modulo}	Retorna lista completa de registros (JSON).
POST	/{modulo}	Crea un nuevo registro; retorna el objeto creado.
GET	/{modulo}/{id}	Retorna un registro especifico por ID.
PUT	/{modulo}/{id}	Actualiza un registro completo.
PATCH	/{modulo}/{id}	Actualiza campos parciales de un registro.

Metodo	Ruta	Descripcion
DELETE	/{modulo}/{id}	Elimina (soft o hard) un registro.
GET	/{modulo}/export/csv	Retorna StreamingResponse con todos los registros en CSV.
GET	/{modulo}/search	Filtra registros por campo y valor (query params).
GET	/{modulo}/count	Retorna el conteo de registros activos (para el dashboard).

4.4 Logica Especial: Modulo de Ordenes

- POST /orders — valida stock disponible antes de crear la orden.
- PATCH /orders/{id} con status=completed descuenta stock de cada producto automaticamente.
- PATCH /orders/{id} con status=cancelled restaura el stock si la orden estaba completada.

4.5 Modulo de Reportes (routers/report_router.py)

Endpoint	Descripcion
GET /reports/sales	Ventas filtradas por rango de fechas, cliente y estado.
GET /reports/inventory	Stock actual vs. minimo; valor total del inventario.
GET /reports/employees	Plantilla por departamento con suma de salarios.
GET /reports/clients	Clientes ordenados por volumen de compra.
GET /reports/sales/export	Descarga CSV del reporte de ventas.
GET /reports/inventory/export	Descarga CSV del inventario.
GET /reports/dashboard	KPIs agregados para el comando bm dashboard.

5. Interfaz CLI — Typer + Rich

La interfaz de usuario completa se implementa en Python puro: Typer define los comandos y argumentos, Rich renderiza tablas, paneles, colores y prompts en la terminal. No existe ningun archivo HTML, CSS ni JavaScript en el proyecto.

5.1 cli.py — Aplicacion Typer Raiz

```
import typer
from commands import (auth_cmd, employee_cmd, product_cmd,
                      client_cmd, order_cmd, report_cmd)

app = typer.Typer(help='Business Manager CLI')
app.add_typer(auth_cmd.app,     name='auth')
app.add_typer(employee_cmd.app, name='employees')
app.add_typer(product_cmd.app,  name='products')
app.add_typer(client_cmd.app,   name='clients')
app.add_typer(order_cmd.app,    name='orders')
app.add_typer(report_cmd.app,   name='reports')

@app.command('dashboard')
def dashboard(): ...

if __name__ == '__main__':
    app()
```

5.2 Patron de Comando CRUD (commands/employee_cmd.py como ejemplo)

```
import typer, httpx
from rich.console import Console
from rich.table  import Table
from rich        import print as rprint
from utils.token import get_token   # lee ~/.bm_token

app     = typer.Typer()
console = Console()
BASE    = 'http://127.0.0.1:8000'

@app.command('list')
def list_employees(
    search: str = typer.Option('', help='Filtrar por nombre o cargo'),
    status: str = typer.Option('all', help='active|inactive|all'),
):
    resp = httpx.get(f'{BASE}/employees',
                     headers={'Authorization': f'Bearer {get_token()}'},
                     params={'search': search, 'status': status})
    data = resp.json()
    table = Table(title='Empleados', style='blue')
    for col in ['ID','Nombre','Cargo','Depto','Salario','Estado']:
```

```
        table.add_column(col)
    for e in data:
        table.add_row(
            e['id'],
            f"{e['first_name']} {e['last_name']}",
            e['position'], e['department'],
            f"CRC {e['salary']:,.0f}", e['status'])
    console.print(table)

@app.command('create')
def create_employee():
    first_name = typer.prompt('Nombre')
    last_name  = typer.prompt('Apellidos')
    email      = typer.prompt('Email')
    position   = typer.prompt('Cargo')
    department = typer.prompt('Departamento')
    salary     = typer.prompt('Salario', type=float)
    hire_date  = typer.prompt('Fecha ingreso (YYYY-MM-DD)')
    resp = httpx.post(f'{BASE}/employees',
                      json=dict(first_name=first_name,
                                last_name=last_name, email=email,
                                position=position, department=department,
                                salary=salary, hire_date=hire_date),
                      headers={'Authorization': f'Bearer {get_token()}'})
    rprint('[green]Empleado creado:[/green]', resp.json())
```

5.3 Tabla Completa de Comandos CLI

Modulo	Comando	Descripcion
auth	bm auth login	Login interactivo con prompt de empresa y contrasena.
auth	bm auth register	Registro interactivo de nueva empresa.
auth	bm auth logout	Cierra la sesion actual; elimina ~/.bm_token.
auth	bm auth status	Muestra la empresa autenticada en un Panel Rich.
--	bm dashboard	KPIs de la empresa activa con paneles y tablas Rich.
employees	bm employees list	Tabla Rich con empleados (filtros opcionales).
employees	bm employees create	Crea empleado con prompts interactivos.
employees	bm employees edit <id>	Edita empleado: muestra valores actuales, permite sobreescribir.
employees	bm employees delete <id>	Elimina con confirmacion: Eliminar? [s/N].
employees	bm employees export	Descarga employees_YYYYMMDD.csv en el directorio actual.
products	bm products list	Tabla Rich; filas en amarillo si stock bajo.
products	bm products create	Crea producto con prompts.

Modulo	Comando	Descripcion
products	bm products edit <id>	Edita producto por ID.
products	bm products delete <id>	Elimina producto con confirmacion.
products	bm products stock <id>	Ajuste rapido de stock (+/- unidades).
products	bm products export	Descarga catalogo a CSV.
clients	bm clients list	Tabla Rich con clientes.
clients	bm clients create	Crea cliente con prompts.
clients	bm clients edit <id>	Edita cliente por ID.
clients	bm clients delete <id>	Elimina cliente con confirmacion.
clients	bm clients history <id>	Historial de ordenes del cliente en tabla Rich.
clients	bm clients export	Descarga clientes a CSV.
orders	bm orders list	Lista ordenes con filtro por estado.
orders	bm orders create	Crea orden interactivamente (cliente + items).
orders	bm orders view <id>	Detalle completo de una orden en panel Rich.
orders	bm orders status <id>	Cambia el estado de una orden.
orders	bm orders delete <id>	Cancela / elimina una orden.
reports	bm reports sales	Reporte de ventas con filtros de fecha y cliente.
reports	bm reports inventory	Reporte de inventario con alertas de stock.
reports	bm reports employees	Plantilla y salarios por departamento.
reports	bm reports clients	Clientes ordenados por volumen de compra.

5.4 Dashboard en Terminal (bm dashboard)

El comando bm dashboard llama a GET /reports/dashboard y renderiza el resultado con Rich sin ningun HTML:

```
from rich.console import Console
from rich.panel   import Panel
from rich.table   import Table
from rich.columns import Columns

@app.command('dashboard')
def dashboard():
    data = httpx.get(f'{BASE}/reports/dashboard',
                     headers={'Authorization': f'Bearer {get_token()}'}).json()
    console.rule('[bold blue]Business Manager Dashboard[/bold blue]')
    kpis = Columns([
        Panel(str(data['employees']),  title='Empleados',    style='green'),
        Panel(str(data['products']),   title='Productos',    style='cyan'),
```

```
        Panel(str(data['clients']),    title='Clientes',     style='magenta'),
        Panel(data['monthly_sales'],   title='Ventas (mes)', style='yellow'),
    ])
    console.print(kpis)
    if data['low_stock']:
        t = Table(title='Stock Bajo', style='yellow')
        t.add_column('Producto'); t.add_column('Stock'); t.add_column('Minimo')
        for p in data['low_stock']:
            t.add_row(p['name'], str(p['stock']), str(p['min_stock']))
        console.print(t)
```

6. Dependencias y Gestion con uv

6.1 pyproject.toml

```
[project]
name            = 'business-manager'
version         = '2.1.0'
requires-python = '>=3.12'

[project.dependencies]
fastapi          = '>=0.115.0'
uvicorn          = { extras = ['standard'], version = '>=0.30.0' }
pydantic         = '>=2.7.0'
pyjwt            = '>=2.9.0'
typer            = '>=0.12.0'
rich             = '>=13.7.0'
httpx            = '>=0.27.0'
python-multipart = '>=0.0.9'

[project.scripts]
bm    = 'cli:app'     # CLI: uv run bm <comando>
serve = 'main:start'  # Servidor: uv run serve
```

6.2 Tabla de Dependencias

Libreria	Version	Capa	Uso
fastapi	>=0.115	Backend	Framework REST; routing, Depends, validacion automatica Pydantic.
uvicorn[standard]	>=0.30	Backend	Servidor ASGI que ejecuta la aplicacion FastAPI.
pydantic	>=2.7	Backend	Modelos de datos; validacion y serializacion JSON.
pyjwt	>=2.9	Backend	Generacion y validacion de tokens JWT.
typer	>=0.12	CLI	Framework de CLI; comandos, opciones y argumentos en Python.
rich	>=13.7	CLI	Tablas, paneles, colores, progreso y prompts en la terminal.
httpx	>=0.27	CLI	Cliente HTTP que la CLI usa para llamar a la API FastAPI.
python-multipart	>=0.0.9	Backend	Parseo de formularios multipart (subida de logo).
json	Built-in	Backend	Lectura y escritura de archivos de datos.
hashlib	Built-in	Backend	SHA-256 para hash de contrasenas.
uuid	Built-in	Backend	Generacion de IDs unicos.
datetime	Built-in	Backend	Manejo y validacion de fechas.

Libreria	Version	Capa	Uso
csv	Built-in	Backend	Exportacion a CSV con StreamingResponse.
os / pathlib	Built-in	Backend	Manejo de rutas de archivos y directorios.
re	Built-in	Backend	Regex para validacion de email y SKU.
threading	Built-in	Backend	Lock para escrituras concurrentes al JSON.

6.3 Comandos uv Esenciales

Instalacion de uv (una sola vez):

```
# macOS / Linux
curl -LsSf https://astral.sh/uv/install.sh | sh

# Windows (PowerShell)
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Configuracion inicial del proyecto:

```
uv sync                      # crea .venv e instala todas las dependencias
```

Ejecutar el servidor y la CLI:

```
# Terminal 1 (servidor)
uv run serve

# Terminal 2 (cliente CLI)
uv run bm auth login
uv run bm dashboard
uv run bm employees list
uv run bm orders create
```

Agregar nuevas dependencias:

```
uv add nombre-paquete
```

7. Flujos de Uso

7.1 Flujo de Inicio (Primera Ejecucion)

- Terminal 1: uv run serve -> FastAPI arranca en http://127.0.0.1:8000.
- Terminal 2: uv run bm auth register -> prompts interactivos para crear la primera empresa.
- uv run bm auth login -> solicita empresa y contrasena; guarda JWT en ~/.bm_token.
- uv run bm dashboard -> muestra KPIs de la empresa en terminal con Rich.

7.2 Flujo CRUD Estandar en Terminal

- uv run bm employees list -> tabla Rich con todos los empleados.
- uv run bm employees list --search 'ventas' -> filtra por nombre o cargo.
- uv run bm employees create -> prompts uno a uno; POST /employees; confirmacion en verde.
- uv run bm employees edit emp_001 -> muestra valores actuales; permite sobreescribirlos; PUT /employees/emp_001.
- uv run bm employees delete emp_001 -> confirmacion: Eliminar? [s/N]; DELETE /employees/emp_001.
- uv run bm employees export -> descarga employees_YYYYMMDD.csv en el directorio actual.

7.3 Flujo de Creacion de una Orden

- uv run bm orders create
- Prompt: Cliente (ID o nombre): -> busqueda parcial muestra opciones en tabla Rich.
- Prompt: Agregar item? [s/N] -> se repite hasta que el usuario diga N.
- Por item: Producto (ID o nombre): + Cantidad: -> subtotal calculado y mostrado.
- Resumen con Panel Rich: items, subtotal, impuesto, total.
- Prompt: Confirmar orden? [s/N] -> POST /orders.

8. Orden de Implementacion Recomendado

Fase	Componentes	Entregable
1	pyproject.toml, uv sync, config.py, core/data_manager.py, utils/	Entorno uv funcional. Capa de datos: leer/escribir JSON desde consola.
2	models/ (5 clases Pydantic)	Modelos validables con: uv run python -c 'from models.employee import Employee...'
3	core/auth.py, routers/auth_router.py, main.py	API de autenticacion funcional. JWT operativo. /docs disponible (desarrollo).
4	commands/auth_cmd.py, cli.py, utils/token.py	Login y registro desde terminal: uv run bm auth login/register.
5	Modulo Empleados completo (router + service + comando Typer + Rich)	Primer CRUD completo en terminal. Sirve de plantilla para los siguientes modulos.
6	Modulos Productos y Clientes (router + service + comando)	CRUD de productos con alertas de stock y clientes con historial.
7	Modulo Ordenes (router + service + comando)	Ordenes con items, calculos automaticos y afectacion de stock.
8	Modulo Reportes + Exportacion CSV	Reportes con filtros en terminal y descarga de CSV.
9	Dashboard CLI, manejo de errores, pruebas, pulido de UX en terminal	Aplicacion lista. uv run bm --help muestra todos los comandos disponibles.

9. Convenciones y Buenas Practicas

9.1 Nomenclatura

- Clases Pydantic: PascalCase -> EmployeeCreate, ProductResponse.
- Funciones y variables: snake_case -> get_all_employees(), current_company.
- Constantes: UPPER_SNAKE -> MAX_LOGIN_ATTEMPTS, DATA_DIR, API_BASE_URL.
- Archivos: snake_case -> employee_router.py, employee_cmd.py.
- IDs en JSON: prefijo + uuid4 parcial -> emp_3f2a1b9c.
- Rutas FastAPI: kebab-case -> /low-stock-alerts, /company-profile.

9.2 Manejo de Errores

- Los routers FastAPI usan HTTPException con status codes apropiados (400, 401, 404, 422).
- Los comandos Typer capturan errores HTTP y los muestran con Rich: console.print('[red]Error:[/red]', mensaje).
- Cada guardado de JSON genera primero un archivo .bak como respaldo.
- Los validadores lanzan ValidationError que los servicios convierten en HTTPException.

9.3 Separacion CLI vs API

- Toda la logica de negocio reside en el backend (services/). La CLI solo hace requests y renderiza.
- La CLI nunca accede directamente a los archivos JSON; siempre pasa por la API.
- El token JWT se almacena en ~/.bm_token; utils/token.py lo lee e inyecta en cada request.

9.4 Documentacion de la API

- FastAPI genera automaticamente Swagger UI en http://localhost:8000/docs (util para desarrollo y pruebas).
- Cada endpoint usa summary= y description= para documentar su comportamiento en Swagger.
- Los modelos Pydantic con Field(description=...) aparecen en el schema JSON automaticamente.

10. Glosario de Terminos

Termino	Definicion
CRUD	Create, Read, Update, Delete: las cuatro operaciones basicas de persistencia.
FastAPI	Framework web moderno de Python, alto rendimiento, basado en ASGI y Pydantic.
Typer	Libreria Python para crear CLIs tipadas; basada en Click y type hints de Python.
Rich	Libreria Python para tablas, paneles, colores y progreso en la terminal (TUI).
httpx	Cliente HTTP sincrono/asincrono de Python; la CLI lo usa para llamar a FastAPI.
uv	Gestor de paquetes y entornos virtuales ultrarapido para Python, escrito en Rust.
Pydantic	Libreria de validacion de datos integrada en FastAPI para requests y responses.
JWT	JSON Web Token: estandar de autenticacion sin estado entre CLI y API.
ASGI	Asynchronous Server Gateway Interface: protocolo async para servidores Python.
TUI	Terminal User Interface: interfaz de usuario que corre en la terminal (Rich).
CLI	Command Line Interface: interfaz de comandos de texto (Typer).
SHA-256	Algoritmo de hash criptografico usado para almacenar contrasenas.
KPI	Key Performance Indicator: metricas clave mostradas en el dashboard.
SKU	Stock Keeping Unit: codigo interno unico por producto.
Soft delete	Marcar un registro como inactivo sin eliminarlo fisicamente del JSON.
Hard delete	Eliminar fisicamente un registro del archivo JSON.
Swagger UI	Interfaz web auto-generada por FastAPI en /docs para explorar la API.
Bearer token	Esquema HTTP donde el JWT se envia en el header Authorization: Bearer <token>.
StreamingResponse	Respuesta FastAPI que envia datos en flujo; usada para descargar CSV.